CHAPTER FOUR: SYSTEMS ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the systems analysis and design of the VulBox Automated Application Security Assessment Portal. It builds on the requirements phase and translates stakeholder requirements into an implementable design that can be handed directly to the implementation team. The chapter covers the overall system description, the data that informed design decisions, the user requirements, functional and non-functional specifications, and detailed architectural and modelling diagrams using both structured approaches (Context DFD, Level 1 DFD, ERD) and object-oriented UML 2.0 approaches (Use Case, Class, Sequence, Activity, State).

All design decisions in this chapter are grounded in the three core architectural principles established during the requirements phase: the pipeline must never terminate early on account of vulnerability findings alone; the system must attempt to exploit every identified weakness before coming to conclusions; and when an attack causes the application to crash, the system must recover automatically and continue testing without human intervention.

4.2 Description of the Designed System

The Automated Application Security Assessment Portal is a web-based Security-as-a-Service (SECaaS) platform designed specifically for student developers, early-career engineers, and small prototype teams who need affordable, integrated security validation but lack the budget or infrastructure for enterprise tools. It enables software developers to submit a GitHub repository link and receive a detailed, evidence-based security assessment report that goes beyond identifying vulnerabilities and shows which ones actually work in practice. The entire system is deliberately designed to run on limited compute resources, targeting lightweight Linux virtual machine environments such as those available on DigitalOcean, making it practical for campus and startup settings.

For the system to work, a developer visits the portal, provides the URL of their repository, and confirms consent that grants the system permission to perform active adversarial testing on their application. The system takes over, clones the repository, uses it to build a Docker container image, and runs the application through three sequential phases of analysis, each of which builds on the results of the last.

The first phase uses Trivy to perform a static scan of the container image, identifying all known CVEs and configuration weaknesses. This phase scans for as many known vulnerabilities and even if it discovers dozens of critical vulnerabilities, the pipeline does not stop there. All findings are logged and used to prioritise the adversarial tests that will run in phase three.

The second phase deploys the application inside a network-isolated Docker sandbox and attaches Falco to the Linux kernel. Falco watches every system call the application makes and fires alerts when it observes behaviour that matches known attack patterns. This phase runs alongside phase three, providing real-time visibility into what is happening inside the container during the attacks.

The third phase runs Atomic Red Team tests mapped to the MITRE ATT&CK framework. A prioritised test queue is built with CVE-related tests run first, followed by the full applicable test suite, which it executes sequentially. If an attack causes the application to crash, the system logs the crash, automatically rebuilds and restarts the container, and continues from the next test. There is no manual intervention required.

Once all tests have run, a structured report is generated that, for every identified vulnerability, records three facts: whether it was found by the static scan (Presence), whether it was successfully exploited by an adversarial test (Exploitability), and whether Falco detected the attack at runtime (Detectability). This scoring gives development teams precisely the information they need to prioritise remediation work.

4.2.1 Key System Features

Web-Based Submission Portal: A clean, accessible front-end where Application Providers submit their GitHub repository URL alongside mandatory testing consent.
Three-Phase Security Engine: Phase 1 (Trivy static scan), Phase 2 (Falco runtime monitoring), Phase 3 (Atomic Red Team adversarial simulation) executed in sequence, non-blocking.
Orchestrator: A central Python-based backend controller that manages the entire pipeline, coordinates all three tools, and enforces the non-blocking and self-healing rules.
Self-Healing Pipeline: Automated container rebuild and restart triggered whenever an adversarial test crashes the application. Testing resumes from the next item in the queue.
Security Matrix Report: A structured, downloadable report scoring every vulnerability on Presence, Exploitability, and Detectability.
MITRE ATT&CK Integration: All adversarial tests are mapped to ATT&CK tactic and technique identifiers, enabling security teams to understand findings in a standardised threat intelligence context.

4.3 Data Analysis and Results

The design of this system was informed by a requirements gathering phase that combined a review of existing security tools, stakeholder interviews with software development teams at Makerere University, and an analysis of publicly available vulnerability data and incident reports from sources including the National Vulnerability Database (NVD) and MITRE ATT&CK.

The central finding from this analysis was that development teams consistently reported a mismatch between what their security scanners told them and what their actual exposure turned out to be. In many cases, scanners flagged critical vulnerabilities that later proved

unexploitable in the specific configuration of the application, while in other cases, lower-severity findings have been dismissed and later became the cause for a successful attack. This pattern, alert fatigue, was identified as the primary problem this system is designed to solve.

These findings shaped three core design rules that are enforced throughout the system: the pipeline must always proceed to the exploitation phase regardless of what the static scan finds; every test result must be correlated with runtime monitoring data; and the final report must make the relationship between vulnerability findings and real-world exploitability clear.

4.4 System User Requirements

The following user requirements were derived directly from stakeholder engagement and the data analysis described in Section 4.3. They are stated in plain terms as the specific capabilities that different users need from the system. The system is designed to cater to one primary user, the application provider.

The Application Provider, typically a student developer, early-career engineer, or startup team member, must be able to submit a GitHub repository URL through a web browser without the need to install any local software, configure any tools, or possess prior knowledge of container security tooling.

The Application Provider must be presented with a clear, plain-language description of what active adversarial testing means before being required to grant consent.
The Application Provider must receive a confirmation notification when their assessment has been queued and a further notification when it has completed.
The Application Provider must be able to view and download the Security Matrix report from the portal. The Application Provider must be able to view the status of a running assessment in real time.
The user requirements informed and shaped the functional and non-functional requirements presented in the following section, ensuring that the system remains suitable for use by its intended users.

4.5 Functional and Non-Functional Requirements

4.5.1 Functional Requirements

FR-01: The system shall accept a GitHub repository URL and a consent confirmation as the inputs to initiate an assessment run.

FR-02: The system shall clone the submitted repository and build a Docker container image from it without modification.

FR-03: The system shall execute a static vulnerability scan against the built container image using Trivy as the default scanner, with Clair supported as a configurable alternative. All findings shall be logged to the database. This phase shall never block or

terminate the pipeline regardless of the severity of findings discovered, and this is the Non-Blocking Rule.

FR-04: The system shall correlate static scan findings with runtime telemetry from Falco and Atomic Red Team test outcomes to distinguish genuine, exploitable vulnerabilities from scanner noise, reducing the effective false positive rate presented to the Application Provider.

FR-05: The system shall deploy the container image to a network-isolated Docker sandbox and attach Falco to the Linux kernel to begin runtime monitoring before any adversarial tests are executed.

FR-06: The system shall build a prioritised queue of Atomic Red Team tests, placing tests related to Trivy-identified CVEs at the front of the queue.

FR-07: The system shall execute each test in the queue in sequence. For each test, the system shall collect any Falco alerts triggered during the test execution and correlate them with the test result.

FR-08: If an adversarial test causes the application to crash, the system shall log the crash event, automatically rebuild and restart the container image, and continue with the next test in the queue. This Self-Healing Pipeline shall not require any human intervention.

FR-09: Upon completion of all tests, the system shall generate a Security Matrix report in which each identified vulnerability is scored against three dimensions: Presence (detected by Trivy), Exploitability (exploited by Atomic Red Team), and Detectability (detected by Falco).

FR-10: The system shall make the Security Matrix report available to the Application Provider through the web portal in both viewable and downloadable (PDF) formats.

FR-11: The system shall enforce role-based access control, distinguishing between Application Provider and System Administrator roles with different permissions.

FR-12: The system shall provide ready-to-use CI pipeline templates in the form of a GitHub Actions workflow file (.yml) and a minimal GitLab CI equivalent, enabling Application Providers to integrate the assessment tool into their existing CI/CD pipeline with no custom configuration required.

4.5.2 Non-Functional Requirements

NFR-01: All communication between the portal and the backend shall be encrypted over HTTPS. The Docker sandbox shall have no outbound internet access. Authentication shall use JWT tokens with configurable expiry. The system shall never execute user-submitted code outside the isolated container environment.

NFR-02: The default pipeline overhead introduced by the security assessment shall not exceed 25% of the application's baseline build time (Myrbakken & Colomo-Palacios, 2017), keeping the static scan phase within the 15-minute tolerance preferred by 82.4% of survey respondents. The portal submission request shall receive a queued confirmation within three seconds. Computationally heavy operations, specifically the Atomic Red

Team adversarial simulation phase, shall run asynchronously, off the critical CI path, so that they do not block or delay the build pipeline itself. The system shall support at least 10 concurrent assessment runs without performance degradation.

NFR-03: The Self-Healing Pipeline shall successfully recover from application crashes with a success rate of at least 95%. The system shall maintain an uptime of 99% during assessment working hours. All assessment data shall be persisted to the database within five seconds of generation.

NFR-04: The portal interface shall be accessible to a developer with no prior experience of container security tooling, without training. All technical terms in the Security Matrix report shall be explained in plain language within the report itself.

NFR-05: The system architecture shall support horizontal scaling of the Orchestrator and sandbox components through containerisation to accommodate increasing numbers of concurrent runs.

NFR-06: The Atomic Red Team test library shall be updatable without requiring changes to core Orchestrator logic. Falco rulesets shall be configurable through the administration interface without system downtime.

NFR-07: The system shall support any standard Docker image as a submission target and shall be deployable on any Linux-based CI runner, with first-class support for GitHub Actions and GitLab CI. The backend and orchestration components shall run on a single DigitalOcean Droplet (minimum 2 vCPU / 4 GB RAM) without requiring Kubernetes or any managed container orchestration service, in line with the project's low-cost design mandate.

4.5.3 System Requirements

The following are the technical requirements that define the minimum and recommended hardware and software for deploying the Vulbox application on Digital Ocean.

Component	Minimum Requirements	Recommended specs
CPU	2vCPU cores	4+ vCPU CORES
RAM	4 GB	8+GB
Storage	40GB SSD	80+GB SSD
Operating System	Ubuntu 20.04 LTS	Ubuntu 22.03 LTS
Docker	Docker 20.10	Docker 24.0+
CI/CD tool	Jenkins 2.300+ or GitHub Actions	Jenkins 2.400+ or GitHub Actions
Database	PostgreSQL 12 or SQLite	PostgreSQL 14+

Bandwidth	1 Tbps unmetered	1Tbps unmetered + CDN(Content Delivery Network)
Network	Public IPv4 + private networking	Dedicated IPv4/IPv6 + VPC(Virtual Private Cloud)
Runtime	Node.js 16.x and Python	Node.js18.x and Python 3.11

4.6 High-Level Architecture of the Designed System

The system follows a three-tier architecture organised into a Presentation Layer, a Logic Layer, and a Data Layer. These tiers interact through well-defined interfaces: the Presentation Layer communicates with the Logic Layer over a REST API, and the Logic Layer reads and writes to the Data Layer using a standard database connector. A separate Integration Layer handles communication between the Orchestrator and the three security tools.

The Presentation Layer is a responsive web application built in React that provides the submission portal, the assessment status dashboard, and the Security Matrix report viewer. It connects to the backend over HTTPS and uses WebSocket connections to push real-time status updates to the Application Provider while an assessment is running.

The Logic Layer is centred on the Orchestrator, a Python application built on the FastAPI framework. The Orchestrator is the single most important component in the system. It manages the full lifecycle of every assessment run, decides upon when to invoke each tool, how to handle tool failures, and when to initiate a container rebuild. It is the central point through which all tools that are used communicate. This design decision is intentional as it ensures that the non-blocking rule and the self-healing logic are enforced in one place.

FastAPI was selected as the backend framework for three reasons that map back to the system's requirements. First of all, FastAPI's native support for Python's asyncio and its BackgroundTasks API allows the Orchestrator to accept a submission request and immediately return a run ID to the portal, while the actual three-phase pipeline executes asynchronously in the background. This is what makes the ≤25% overhead non-functional requirement achievable without blocking the CI pipeline. Second, FastAPI is lightweight and starts in under a second on a DigitalOcean Droplet, meaning it imposes negligible overhead on the limited compute resources available. Third, it automatically generates OpenAPI documentation for all endpoints, which simplifies integration testing and makes it easier for future contributors to understand the REST interface without reading through the source code. The combination of async execution, low resource utilisation, and automatic documentation makes FastAPI the most appropriate framework for this specific use case among Python web frameworks.

The Data Layer uses a PostgreSQL database while deployed, with SQLite supported for development and testing. The database stores all assessment runs, Trivy findings, Falco alerts, Atomic Red Team test results, and the final Security Matrix entries. All findings are immutable once written, and the database is an audit log as much as it is an operational store.

4.6.1 Hosting and Compute Environment

The system is hosted on DigitalOcean, chosen specifically because it aligns with the project's low-cost design mandate and the resource profile of the target user base. The production environment runs on a DigitalOcean Droplet (2 vCPU, 4 GB RAM, 80 GB SSD, Ubuntu 22.04 LTS). This configuration is sufficient to run the FastAPI backend, the

PostgreSQL database, the Docker Engine (for building and sandboxing target containers), and a Falco sidecar simultaneously. DigitalOcean's flat-rate pricing, which is typically under USD 24 per month for this tier, is relatively cheap to set up for the development team. This makes it easy to make the system accessible to students at a low price thus meeting the price requirement. The Docker sandbox used for adversarial testing runs as a separate network-isolated container on the same Droplet, with iptables rules blocking all outbound traffic from the sandbox network. For scaling, DigitalOcean Droplet snapshots allow the environment to be replicated quickly if concurrent assessment load increases.

The Integration Layer consists of three lightweight adapter modules, one for each security tool. The Trivy Adapter wraps the Trivy CLI and parses its JSON output into Finding objects. The Falco Adapter attaches Falco as a sidecar to the target container and streams its alert output to the Orchestrator. The ART Adapter manages the Atomic Red Team test library and executes individual tests by invoking the appropriate scripts via the container runtime.

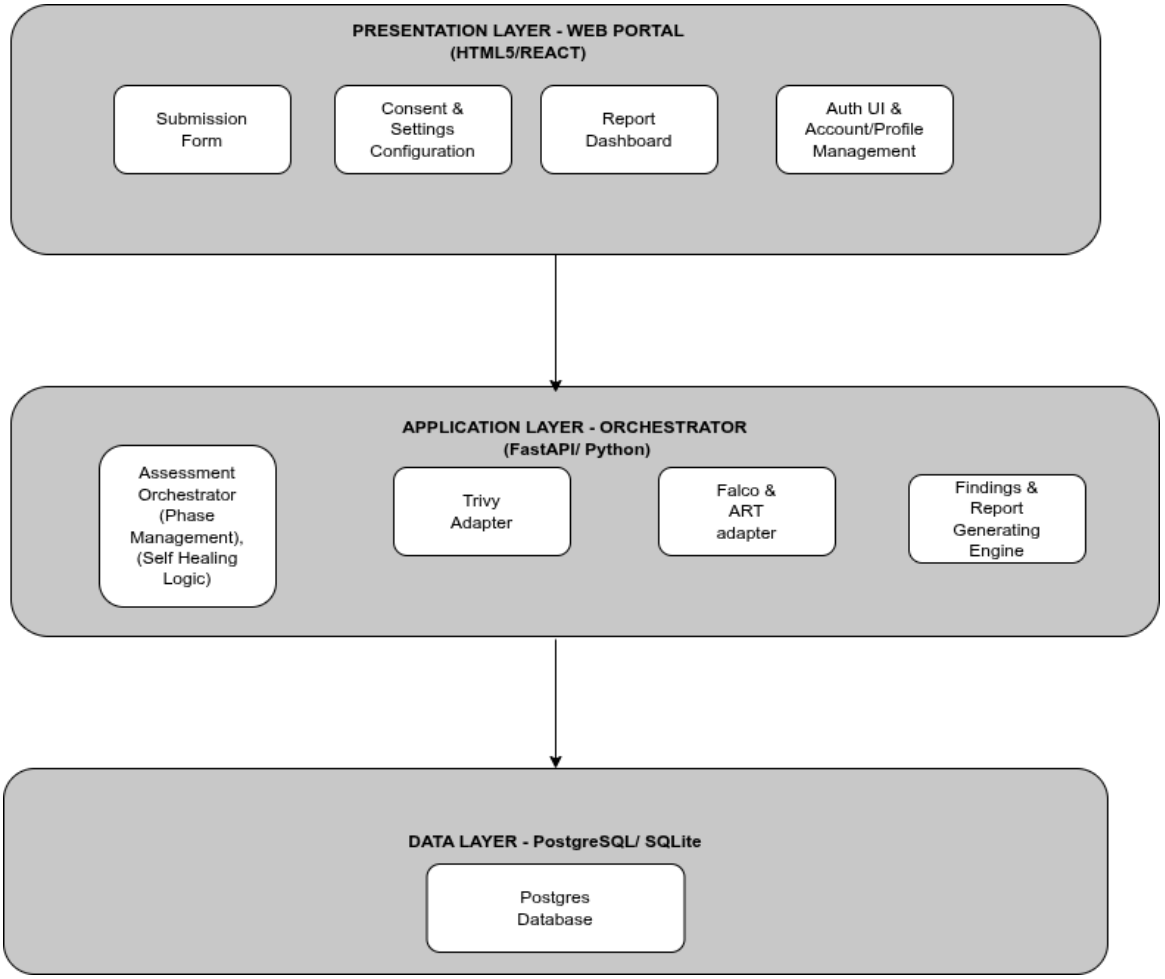

Figure 4.6.1-1: System Architecture Diagram - Three-Tier Layered Architecture

4.7 Process Design - Activity

The process design is illustrated in Figure 4-8 (the Activity Diagram) and described here. The flow is divided into five stages:

Portal Interaction, Phase 1, Phase 2, Phase 3, and Reporting.

The process begins when the Application Provider visits the portal, enters their GitHub repository URL, and ticks the consent checkbox. The portal validates the input and passes it to the Orchestrator, which immediately queues the assessment and responds with a confirmation and a run identifier that the Application Provider can use to track progress.

The Orchestrator then clones the repository, builds a Docker image, and hands control to Phase 1. Trivy scans the image and returns a list of findings. It does not pause the pipeline here due to its findings but continues onto the next phase. This is the Non-Blocking Rule, and it is a key feature that differs from a conventional scanner.

Phase 2 deploys the application to a network-isolated sandbox and attaches Falco. The Orchestrator also builds the ART test queue and gives tests related to the CVEs found in Phase 1 as the priority. Once Falco confirms it is active and monitoring, Phase 3 begins.

Phase 3 is a loop. For each test in the queue, the Orchestrator calls the ART Adapter to execute the test, waits for the result, and then asks the Falco Adapter for any alerts that fired during the test. If the application crashed during the test, the crash is logged, the container is rebuilt and restarted (the Self-Healing Pipeline), and the loop continues with the next test. If the application survives, the result and the Falco alerts are correlated and stored as a Matrix Entry. When the queue is empty, the loop exits.

Finally, the Reporting phase detaches Falco, destroys the sandbox, compiles all Matrix Entries into the Security Matrix report, and makes it available to the Application Provider on the portal.

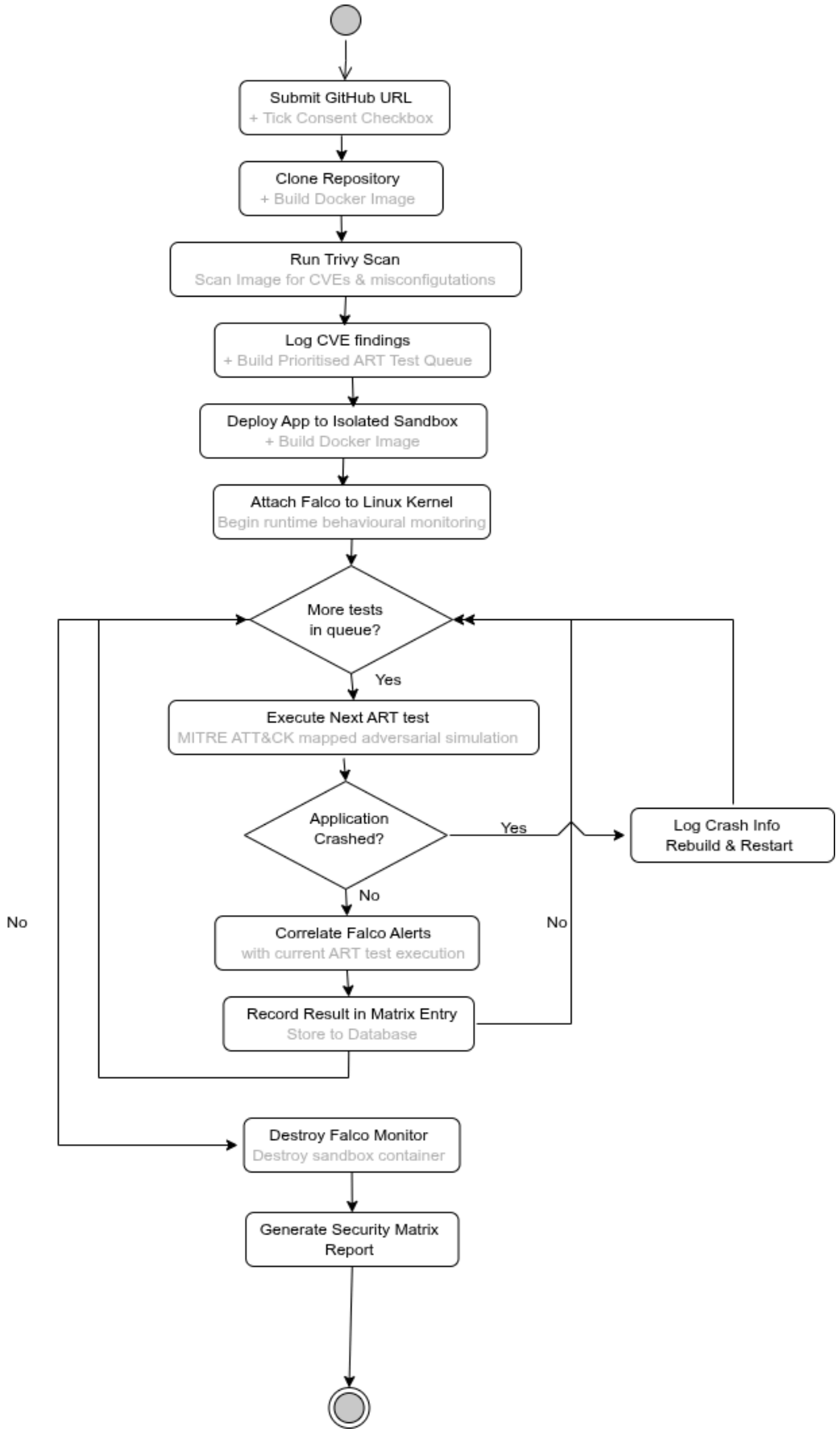

Figure 4-8: Activity Diagram - Non-Blocking Three-Phase Pipeline with Self-Healing Loop

4.8 The Context Diagram (DFD Level 0)

The Context Diagram provides the highest-level view of the system, showing it as a single process surrounded by its external entities and the data flows between them. At this level of abstraction, the internal structure of the system is hidden; what matters is understanding who interacts with the system and what they give to and receive from it.

External Entities

Application Provider: The developer or security team member who submits a repository for assessment. They provide a GitHub URL and testing consent, and they receive the completed Security Matrix report.
GitHub: The source code hosting platform from which the system clones the submitted repository. The system sends a clone request and receives the repository source code.
Trivy: The static application security testing(SAST) used by the system. The system sends the Docker image, and it provides the Common Vulnerabilities and Exposures (CVEs) that it finds.

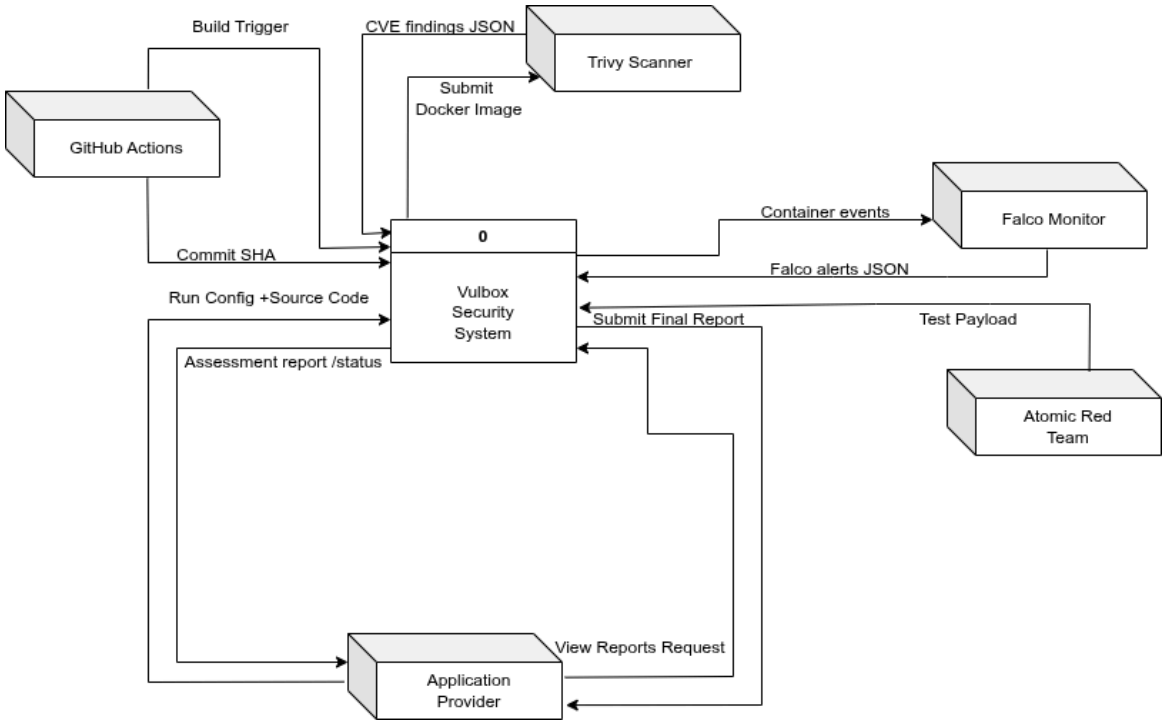

Figure 4.8-1: Context Diagram (DFD Level 0)

4.9 The Use Case Diagram

The Use Case Diagram defines the system's actors and the interactions each actor initiates. One actor is identified, and it is the Application Provider. The Application Provider is the primary actor, and all the core values of the system are delivered to them through their interactions with it.

Application Provider: The developer, security engineer, or any other person who owns or is authorised to test the application being submitted. This is the primary actor.

Primary Use Cases

Submit Assessment Run: The Application Provider's primary interaction with the system. It includes the provision of the GitHub repository link for testing.
Configure the running configuration: The Application Provider decides what tests to run according to their preference and the focal point of the project they are testing.
Grant Testing Consent [<<include>>]: An interaction of Run Atomic Validation. The Application Provider explicitly authorises active adversarial testing of their application.
Run Trivy Scan: Trivy scans the container image and logs findings without blocking the pipeline.
Build Docker Image: The application undergoing testing is cloned into a Docker image.
Deploy to Sandbox: The built image is then deployed in an isolated environment (sandbox) where the testing is to be carried out.
Run Atomic Validation: Atomic Red Team tests are executed in a prioritised sequence.
Monitor Runtime: Falco is attached for runtime monitoring.
Correlate findings: Results from Trivy and Falco, which monitored the attacks performed on the application, are cross-referenced to find results which are stored in the database.
Generate Report: The Application Provider retrieves and reviews their completed Security Matrix.

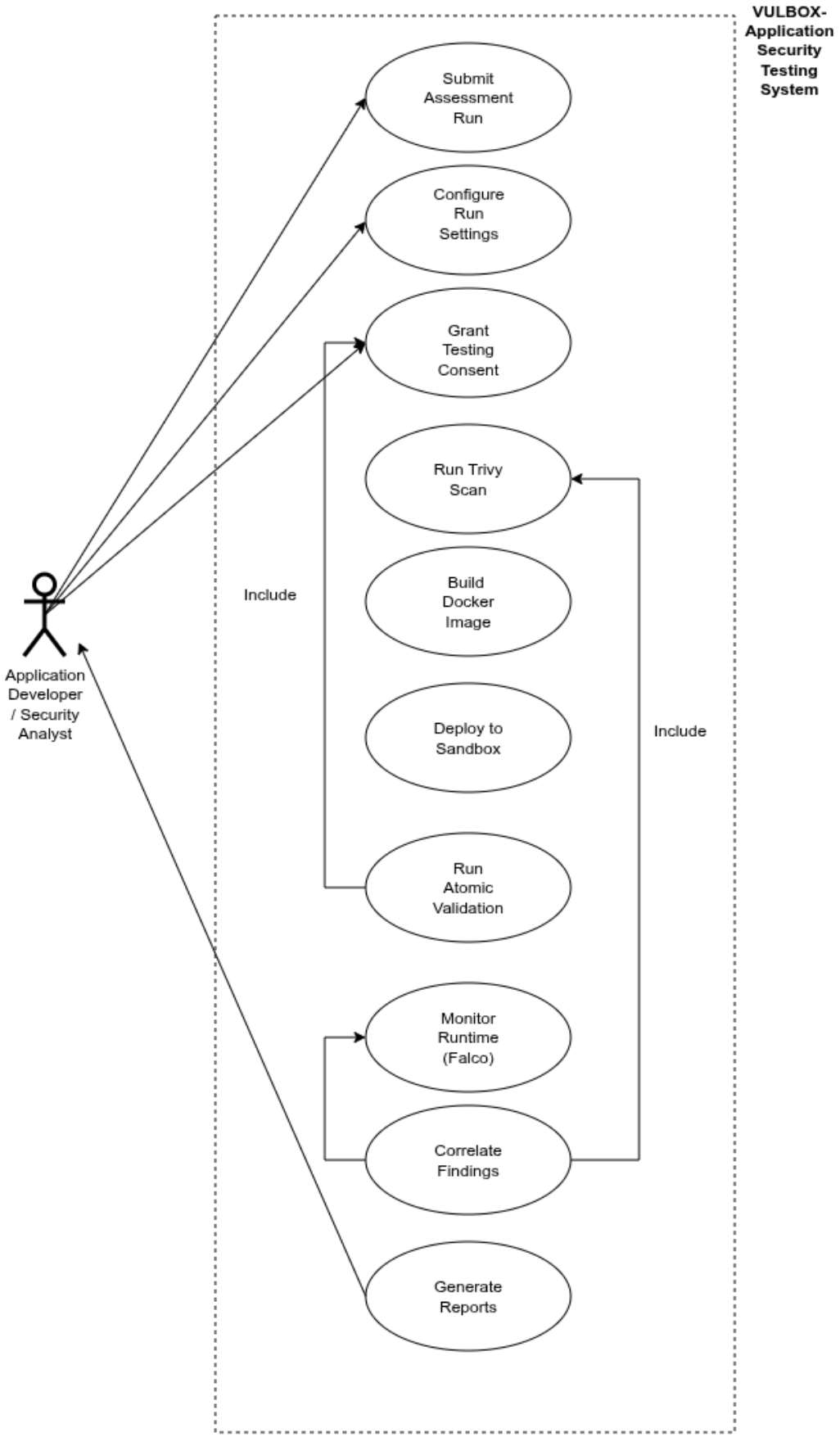

Figure 4-6: Use Case Diagram

4.10 Entity Relationship Diagram

The database design reflects the three-phase structure of the system. Every piece of data generated during an assessment is mapped with a single AssessmentRun record, making it possible to retrieve the history of any assessment at any time.

Entities

AssessmentRun: Attributes: [runId (PK), repoUrl, consentGranted, status, createdAt, completedAt, submittedBy]. The root record for every assessment. All other entities reference this through a foreign key on runId.

TrivyFinding: Attributes: [findingId (PK), runId (FK), cveId, severity, packageName, description, fixAvailable]. One record per CVE or misconfiguration identified during the static scan. A single run typically produces multiple findings.

ARTTestResult: Attributes: [testResultId (PK), runId (FK), mitreTestId, exploited, crashOccurred, executedAt]. One record per Atomic Red Team test executed. The exploited and crashOccurred Boolean fields are the primary data points for the Security Matrix.

FalcoAlert: Attributes: [alertId (PK), testResultId (FK), ruleTriggered, severity, syscallContext, timestamp, detected]. One record per Falco alert. Associated with a specific test result, enabling direct correlation between attack activity and monitoring response.

SecurityMatrixEntry: Attributes: [entryId (PK), runId (FK), findingId (FK), testResultId (FK), isPresent, isExploitable, isDetectable, mitreTacticId, riskScore]. The central output entity. Joins finding, test result, and alert data into the three-dimensional Security Matrix entry.

Figure 4-7: Entity Relationship Diagram (ERD)

4.11 Dynamic Modelling

Dynamic modelling describes how the system behaves over time, especially how objects interact in sequence, how control flows through the system during key operations, and how individual objects change state during their lifetime. This section presents four types of dynamic diagrams: sequence, collaboration, and state.

4.11.1 Sequence Diagram

The Sequence Diagram models the time-ordered message passing between system components during a complete assessment run. The objects that take part during the assessment are the Application Provider, Dashboard, Fast API, GitHub Actions, Scanners, SQLite and the LLM service, with time progression represented downward.

The sequence begins with the Application Provider creating an account an uploading github repository link with the preferred run configurations.

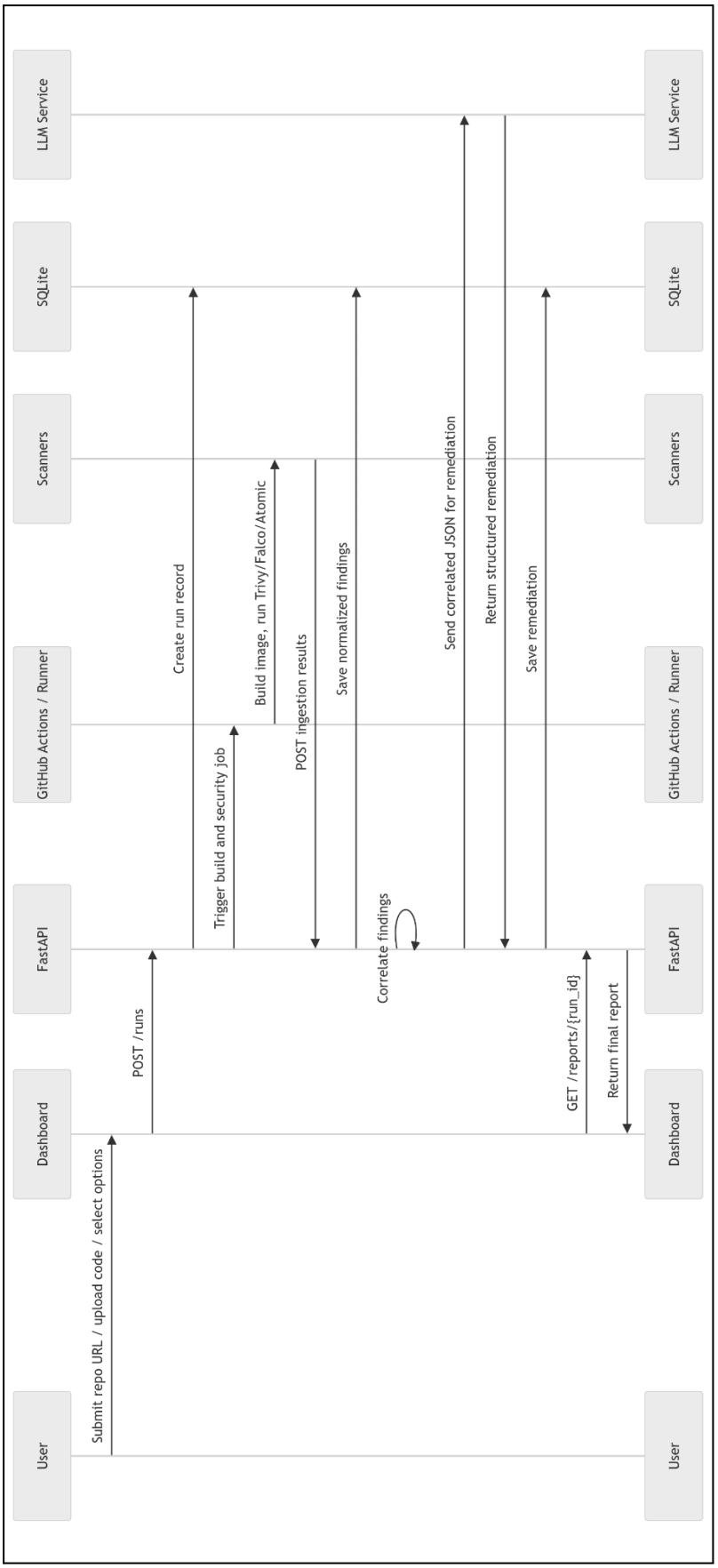

Figure 4-9: Sequence Diagram

4.11.2 State Transition Diagram

The State Transition Diagram models the lifecycle of a single AssessmentRun object from its creation to completion. An AssessmentRun passes through a total of seven possible states: SUBMITTED, BUILDING, SCANNING, TESTING, REBUILDING, REPORTING, and COMPLETE, with FAILED as an exit state reached only if the build phase fails (meaning no image could be constructed from the repository).

The most important feature of this diagram is the REBUILDING state and its transitions. When the TESTING state detects a crash (the crash() event), the object transitions to REBUILDING. Once the rebuild is complete (the rebuild() event), it transitions back to TESTING and not forward to REPORTING, and not to FAILED. This loop between TESTING and REBUILDING is the formal representation of the Self-Healing Pipeline and confirms that the system is designed to recover from crashes and continue, not to give up.

The FAILED state is only reachable from BUILDING, not from any later state. This is an intentional design decision: if the application cannot be built into a container at all, there is nothing to test. But once a container exists and testing has begun, no subsequent failure, including application crashes, should terminate the assessment.

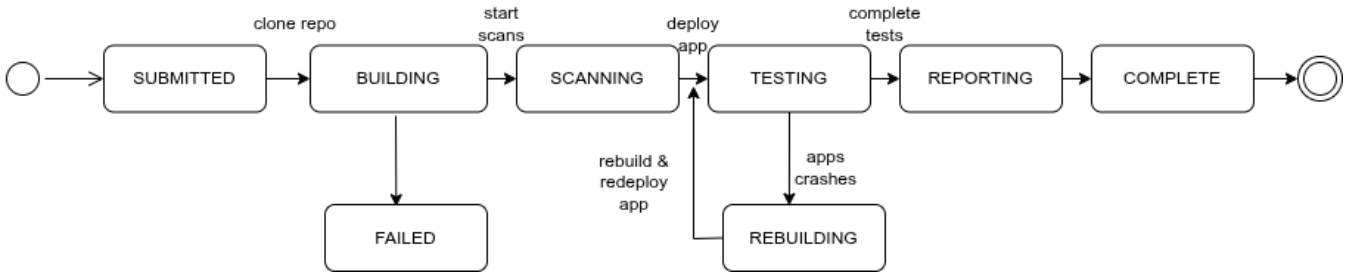

Figure 4.11.2-1: State Transition Diagram - AssessmentRun Lifecycle

4.12 Class Diagram (CoCIS UML 2.0 Standard)

The Class Diagram presents the object-oriented structure of the system, showing all major classes and their attributes and methods.

4.12.1 Visibility Notation

The following visibility markers are used throughout the diagram:

+ (plus sign): Public. The member is accessible from any other class.
- (minus sign): Private. The member is only accessible within the defining class.
 # (hash): Protected. The member is accessible within the class and its subclasses.

4.12.2 Core Classes

AssessmentRun <<entity>>: The data object representing a single assessment run. It is created by the Portal layer when a submission is received and updated by the Orchestrator as the run progresses. The consentGranted: Boolean attribute is particularly important,

and the system's submission logic must verify that this is true before any processing begins.

Orchestrator <<controller>>: The central control class of the system. It holds references to the test queue and findings list, and exposes the core pipeline methods: startAssessment(), cloneAndBuild(), rebuildContainer(), and generateReport(). The rebuildContainer() method is the programmatic representation of the Self-Healing Pipeline. All its methods are public because they are called from the Portal layer or from within the pipeline itself.

TrivyScanner <<scanner>>: Encapsulates all interaction with the Trivy CLI. Its most significant method is scan(imageRef: String): List<Finding>, which is the entry point to Phase 1. The isBlocking(): Boolean method always returns false, implementing the Non-Blocking Rule as a code-level constraint rather than a configuration option. This ensures the rule cannot be accidentally bypassed.

FalcoMonitor <<monitor>>: Manages the Falco process lifecycle and alert collection. The attach() and detach() methods start and stop monitoring. The correlate() method links a specific Falco Alert to the test that was running when it fired, producing a CorrelationResult that becomes part of the Matrix Entry.

AtomicRedTeamSimulator <<simulator>>: Manages the test library and execution queue. The buildQueue() method accepts the list of Trivy Findings and returns an ordered list of test identifiers, with CVE-related tests at the front. The executeTest() method runs a single test and returns a TestResult.

SecurityReport <<report>>: The output object of the entire pipeline. It contains a list of MatrixEntry objects, one per vulnerability. The exportPDF() and getSummaryStats() methods support the report delivery function of the Portal.

DockerManager <<infrastructure>>: Abstracts all Docker Engine operations. The rebuildAndRestart() method is called by the Orchestrator when a crash is detected and is the technical implementation of the Self-Healing Pipeline.

Figure 4-6-cls: Class Diagram (REVISED - CoCIS UML 2.0 Standard)

4.13 The Security Matrix (System Output Design)

The Security Matrix is the most important output of the system. Its design deserves its own section because the structure of the matrix determines what value the system delivers to the Application Provider.

For every security finding identified during an assessment, the Security Matrix provides three data points:

- **Presence:** Was the flaw identified during the static scan by Trivy? This answers the question: "Is this vulnerability in my code?" A positive answer confirms the finding is real and the CVE database entry applies to a reliance or framework in the submitted application.

- **Exploitability:** Could an attacker actually use this flaw to compromise the system? This is the most important aspect. It answers the question: "If a real attacker found this, could they do anything with it?" A finding that is Present but not Exploitable is a low-priority remediation item. A finding that is both Present and Exploitable demands urgent attention.
- **Detectability:** Did the runtime monitoring catch the attack when it was executed? This answers the question: "If an attacker exploited this right now, would anyone know?" A finding that is vulnerable but not Detectable represents not just a vulnerability but a monitoring gap which makes it two problems to fix, not one.

By combining these three factors, the Security Matrix is significantly more actionable than a traditional vulnerability report. An Application Provider who receives a list of 200 CVEs has no clear starting point. An Application Provider who receives a matrix that shows, for example, that 12 of those CVEs are exploitable, 8 of those exploitable CVEs are also undetectable, and 5 of those have a MITRE ATT&CK tactic of Credential Access that Application Provider has a clear, prioritised improvement plan.

4.14 Key Design Decisions and Justification

4.14.1 The Non-Blocking Rule

The decision to make Phase 1 non-blocking and to always proceed to Phase 2 and Phase 3 regardless of how serious the Trivy findings are, which is perhaps the most contrary design decision in the system. Conventional CI/CD pipelines are built around the opposite principle: where a build is stopped immediately if critical issues are found during static analysis

This system takes a different position because blocking on static findings alone produces false certainty. A "Critical" CVSS score in the NVD database tells you how bad a vulnerability is in the abstract; it tells you nothing about whether your specific application, in its specific configuration, running on its specific infrastructure, can actually be exploited. Without running the concurrent simulation phase, the system cannot answer the only question that truly matters: "Can an attacker actually compromise my system right now?"

The Non-Blocking Rule is therefore not a relaxation of security standards but an improvement of them. It insists that the only meaningful proof of vulnerability is evidence of exploitability, not a CVSS score.

A good example of the downside of the blocking approach is illustrated by the RENU (Research and Education Network, Uganda) CI/CD pipeline case study collected during the project's requirements gathering phase (Appendix II). In that pipeline, a Trivy static scan sits on the path and routes to "Fail Build & Notify" when vulnerabilities are found which is exactly the pattern this system is designed to move beyond. The RENU pipeline never reaches runtime monitoring or exploit validation for any image that triggers a Trivy finding, meaning it has no mechanism to distinguish a theoretical CVE from a genuinely exploitable weakness. This system treats the RENU pipeline as the baseline problem

statement: good intentions implemented with a blocking gate produce alert fatigue and delays deployments without producing better security outcomes.

4.14.2 The Self-Healing Pipeline

The Self-Healing Pipeline, the automated container rebuild triggered by application crashes, was designed in response to a practical observation: some of the most important security findings are the ones that crash the application. A crash may indicate an exploitable buffer overflow, a denial-of-service attack, or a harmful user input. If the pipeline stops when a crash occurs, those later tests never run, and the system produces an incomplete picture of what the application's security really is.

The Self-Healing Pipeline ensures that a crash is treated as a data point, not an error. It is logged, the container is rebuilt, and the next test runs. The rebuild process produces a fresh, clean instance of the application, ensuring that a crash in test N does not affect the validity of test N+1.

4.14.3 Orchestrator-Centric Architecture

The decision to route all tool communication through the Orchestrator, rather than allowing tools to communicate directly with each other,and this was made for three reasons. First, it makes the pipeline logic verifiable: everything the system does is visible in the Orchestrator's execution log. Second, it makes the pipeline testable: the Orchestrator can be tested alone by substituting test versions of the tool adapters. Third, it makes the system maintainable: if a security tool is updated and its interface changes, only the matching adapter needs to change; therefore, the Orchestrator logic is not affected.

REFERENCES

Aqua Security. (2023). Trivy: Vulnerability Scanner for Containers and Other Artifacts. Retrieved from https://github.com/aquasecurity/trivy

Falco Project. (2023). Falco: Cloud Native Runtime Security. The Falco Authors. Retrieved from https://falco.org

Red Canary. (2023). Atomic Red Team: Library of Tests Mapped to the MITRE ATT&CK Framework. Retrieved from https://github.com/redcanaryco/atomic-red-team

MITRE Corporation. (2023). MITRE ATT&CK Framework: Adversarial Tactics, Techniques and Common Knowledge. Retrieved from https://attack.mitre.org

National Institute of Standards and Technology. (2023). National Vulnerability Database (NVD). Retrieved from https://nvd.nist.gov

Shostack, A. (2014). Threat Modeling: Designing for Security. Wiley.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps Handbook. IT Revolution Press.

Docker Inc. (2023). Docker Documentation: Security in Docker. Retrieved from https://docs.docker.com/engine/security/

Myrbakken, H., & Colomo-Palacios, R. (2017). *DevSecOps: A Systematic Literature Review.* In Systems, Software and Services Process Improvement: 24th European Conference, EuroSPI 2017. Springer.